实 验 报 告

实验名称：实验三 Hebb 学习

课程名称：认知科学与类脑识别

实验地点：K2

实验日期：2024.10.9

班 级：2022 级人工智能班

姓 名：薛润泽

学 号：202200130077

一 实验目的：

加深对 Hebb 学习模型的理解，能够使用 Hebb 学习模型解决简单问题

二 实验环境：

Windows

三 实验内容：

根据 Hebb 学习模型的相关知识，使用 Python 语言实现一个简单 Hebb 学习模型。

四 实验步骤：

1. 实验程序：

（1）构建数字 0,1,2 数字矩阵，同时构建带有固定噪声的 0,1,2 数字矩阵

```python
import numpy as np
import matplotlib.pyplot as plt
digit_0 = np.array([
    [1, 1, 1, 1, 1],
    [1, -1, -1, -1, 1],
    [1, -1, -1, -1, 1],
    [1, -1, -1, -1, 1],
    [1, -1, -1, -1, 1],
    [1, 1, 1, 1, 1]
])

digit_1 = np.array([
    [-1, -1, 1, -1, -1],
    [-1, 1, 1, -1, -1],
    [-1, -1, 1, -1, -1],
    [-1, -1, 1, -1, -1],
    [-1, -1, 1, -1, -1],
    [-1, 1, 1, 1, -1]
])

```

```
digit_2 = np.array([
        [-1,  1,  1, -1, -1],
        [ 1, -1, -1,  1, -1],
        [-1, -1, -1,  1, -1],
        [-1, -1,  1, -1, -1],
        [-1,  1, -1, -1, -1],
        [ 1,  1,  1,  1,  1]
])
noise_digit_0 = np.array([
    [1, 1, 1, 1, 1],
    [1, -1, -1, -1, 1],
    [1, -1, 1, -1, 1],
    [1, 1, -1, -1, 1],
    [1, -1, -1, -1, 1],
    [1, 1, 1, 1, 1]
])

noise_digit_1 = np.array([
    [-1, -1, 1, -1, -1],
    [-1, 1, 1, -1, -1],
    [-1, -1, 1, -1, 1],
    [-1, -1, 1, -1, -1],
    [-1, -1, 1, -1, -1],
    [-1, 1, 1, 1, -1]
])
```

（2）构建阶跃函数以及训练函数

```
digit_0_list = digit_0.reshape(-1)
digit_1_list = digit_1.reshape(-1)
digit_2_list = digit_2.reshape(-1)
def step_f2(x):
    if x > 0:
        return 1
    else :
        return -1
def calc(w1, x):
    return np.dot(w1, x)
def train(epoch,lr, w1, d1, d2, d3):
    d1l = d1.reshape(-1)
    d2l = d2.reshape(-1)
    d3l = d3.reshape(-1)
    for k in range(epoch):
        for j in range(30): #枚举列向量
            for i in range(30):
                w1[i][j] = w1[i][j] + lr * d1l[i] * d1l[j]
                w1[i][j] = w1[i][j] + lr * d2l[i] * d2l[j]
                w1[i][j] = w1[i][j] + lr * d3l[i] * d3l[j]
    return w1

def pre(w1, x1):
```

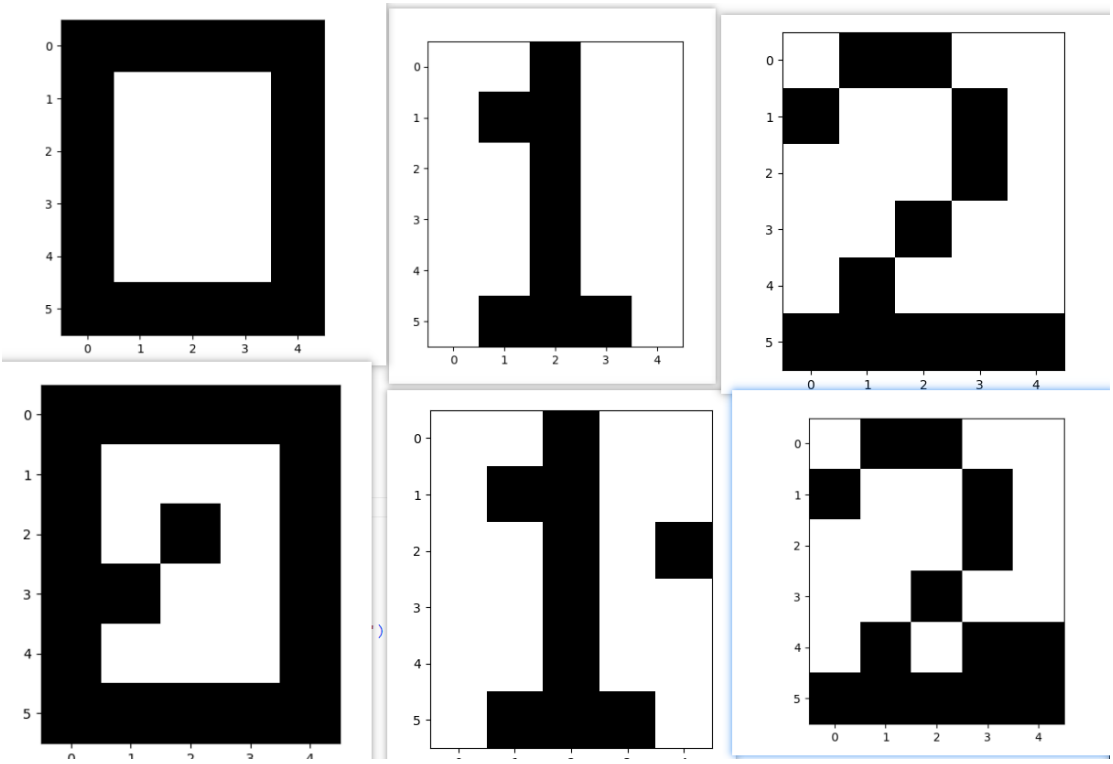
（3）构建预测函数，采用训练 100 轮，步长 0.001 进行模型训练。

并对加噪声的 0,1,2 进行重新预测，并画图。

```
79
80  def pre(w1, x1):
81      x1l = x1.reshape(-1)
82      res = []
83      for i in range(30):
84          # res.append(np.dot(w1[i], x1l))
85          res.append(step_f2(np.dot(w1[i], x1l)))
86      res = np.array(res)
87      res = res.reshape(6, 5)
88      return res
89  w = train(100, 0.001, w, digit_0, digit_1, digit_2)
90  rem = pre(w, noise_digit_2)
91  print(rem)
92  plt.imshow(digit_2, cmap='binary',interpolation = 'nearest')
93  plt.show()
94  plt.imshow(noise_digit_0, cmap='binary',interpolation = 'nearest')
95  plt.show()
96  plt.imshow(rem, cmap='binary',interpolation = 'nearest')
97  plt.show()
98
```

2. 程序实验结果：

（1）原始矩阵构建结果（上方）、噪声矩阵构建结果（下方）

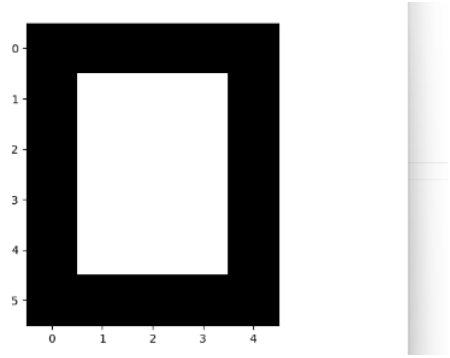


（2）

预测结果如下：

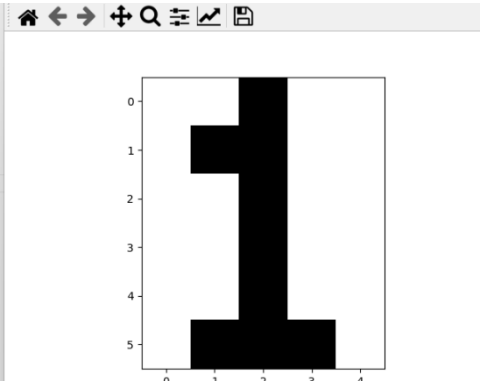
```
    res = np.array(res)
    res = res.reshape(6, 5)
    return res
w = train(100, 0.001, w, digit_0, digit_1, digit_2)
rem = pre(w, noise_digit_0)
print(rem)
plt.imshow(digit_0, cmap='binary',interpolation = 'nearest')
plt.show()
plt.imshow(noise_digit_0, cmap='binary',interpolation = 'nearest')
plt.show()
plt.imshow(rem, cmap='binary',interpolation = 'nearest')
plt.show()

```

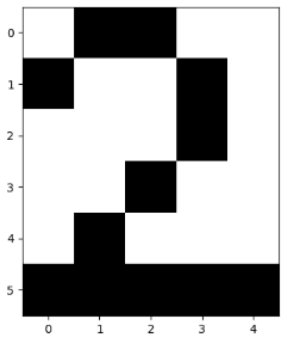


```
        # res.append(np.dot(w1[i], x11))
        res.append(step_f2(np.dot(w1[i], x11)))
    res = np.array(res)
    res = res.reshape(6, 5)
    return res
w = train(100, 0.001, w, digit_0, digit_1, digit_2)
rem = pre(w, noise_digit_1)
print(rem)
plt.imshow(digit_1, cmap='binary',interpolation = 'nearest')
plt.show()
plt.imshow(noise_digit_1, cmap='binary',interpolation = 'nearest')
plt.show()
plt.imshow(rem, cmap='binary',interpolation = 'nearest')
plt.show()

```



```
    x11 = x1.reshape(-1)
    res = []
    for i in range(30):
        # res.append(np.dot(w1[i], x11))
        res.append(step_f2(np.dot(w1[i], x11)))
    res = np.array(res)
    res = res.reshape(6, 5)
    return res
w = train(100, 0.001, w, digit_0, digit_1, digit_2)
rem = pre(w, noise_digit_2)
print(rem)
plt.imshow(digit_2, cmap='binary',interpolation = 'nearest')
plt.show()
plt.imshow(noise_digit_2, cmap='binary',interpolation = 'nearest')
plt.show()
plt.imshow(rem, cmap='binary',interpolation = 'nearest')
plt.show()
```



3. 分析和改进：

可以看到，0,1,2 三张添加了噪声的数据修复成功，说明模型效果好

五 实验小结：

Hebb 学习法是一个类似于 Hopfield 模型训练方法的学习方法。训练时有两种训练方法，如果使用真实值进行训练，不需要考虑初始化问题，如果使用预测值进行训练，则初始化需要做到矩阵内元素数字随机且符号相同。

代码附录：

```
import numpy as np
import matplotlib.pyplot as plt
digit_0 = np.array([
    [1, 1, 1, 1, 1],
    [1, -1, -1, -1, 1],
```

```
    [1, -1, -1, -1, 1],
    [1, -1, -1, -1, 1],
    [1, -1, -1, -1, 1],
    [1, 1, 1, 1, 1]
])

digit_1 = np.array([
    [-1, -1, 1, -1, -1],
    [-1, 1, 1, -1, -1],
    [-1, -1, 1, -1, -1],
    [-1, -1, 1, -1, -1],
    [-1, -1, 1, -1, -1],
    [-1, 1, 1, 1, -1]
])

digit_2 = np.array([
        [-1,  1,  1, -1, -1],
        [ 1, -1, -1,  1, -1],
        [-1, -1, -1,  1, -1],
        [-1, -1,  1, -1, -1],
        [-1,  1, -1, -1, -1],
        [ 1,  1,  1,  1,  1]
```

```
])
noise_digit_0 = np.array([
    [1, 1, 1, 1, 1],
    [1, -1, -1, -1, 1],
    [1, -1, 1, -1, 1],
    [1, 1, -1, -1, 1],
    [1, -1, -1, -1, 1],
    [1, 1, 1, 1, 1]
])

noise_digit_1 = np.array([
    [-1, -1, 1, -1, -1],
    [-1, 1, 1, -1, -1],
    [-1, -1, 1, -1, 1],
    [-1, -1, 1, -1, -1],
    [-1, -1, 1, -1, -1],
    [-1, 1, 1, 1, -1]
])

noise_digit_2 = np.array([
        [-1,  1,  1, -1, -1],
        [ 1, -1, -1,  1, -1],
```

```
        [-1, -1, -1,  1, -1],
        [-1, -1,  1, -1, -1],
        [-1,  1, -1, 1, 1],
        [ 1,  1,  1,  1,  1]
])

w = np.array(np.full((30, 30), 0.01))

digit_0_list = digit_0.reshape(-1)
digit_1_list = digit_1.reshape(-1)
digit_2_list = digit_2.reshape(-1)
def step_f2(x):
      if x > 0:
            return 1
      else :
            return -1
def calc(w1, x):
    return np.dot(w1, x)
def train(epoch,lr, w1, d1, d2, d3):
    d1l = d1.reshape(-1)
    d2l = d2.reshape(-1)
    d3l = d3.reshape(-1)
```

```
    for k in range(epoch):
        for j in range(30): #枚举列向量
            for i in range(30):
                w1[i][j] = w1[i][j] + lr * d1l[i] *
d1l[j]
                w1[i][j] = w1[i][j] + lr * d2l[i] *
d2l[j]
                w1[i][j] = w1[i][j] + lr * d3l[i] *
d3l[j]
    return w1

def pre(w1, x1):
    x1l = x1.reshape(-1)
    res = []
    for i in range(30):
        # res.append(np.dot(w1[i], x1l))
         res.append(step_f2(np.dot(w1[i], x1l)))
    res = np.array(res)
    res = res.reshape(6, 5)
    return res
w = train(100, 0.001, w, digit_0, digit_1, digit_2)
rem = pre(w, noise_digit_2)
```

```
print(rem)
plt.imshow(digit_2, cmap='binary',interpolation =
'nearest')
plt.show()
plt.imshow(noise_digit_2,
cmap='binary',interpolation = 'nearest')
plt.show()
plt.imshow(rem, cmap='binary',interpolation =
'nearest')
plt.show()
```